

Luring Snake Out

Gameplay Introduction

```
ros2 run dofbot_info dofbot_server # Start server node - this node only needs to be started once!
```

3) Luring Snake Out gameplay path:

```
/home/yahboom/dofbot_ws/src/dofbot_snake_follow/scripts/引蛇出洞  
snake_follow_color.ipynb
```

(1) Start the code block, the interface shown in the figure is displayed. Click any color button to select a color. The selected color is displayed in the middle above the image.

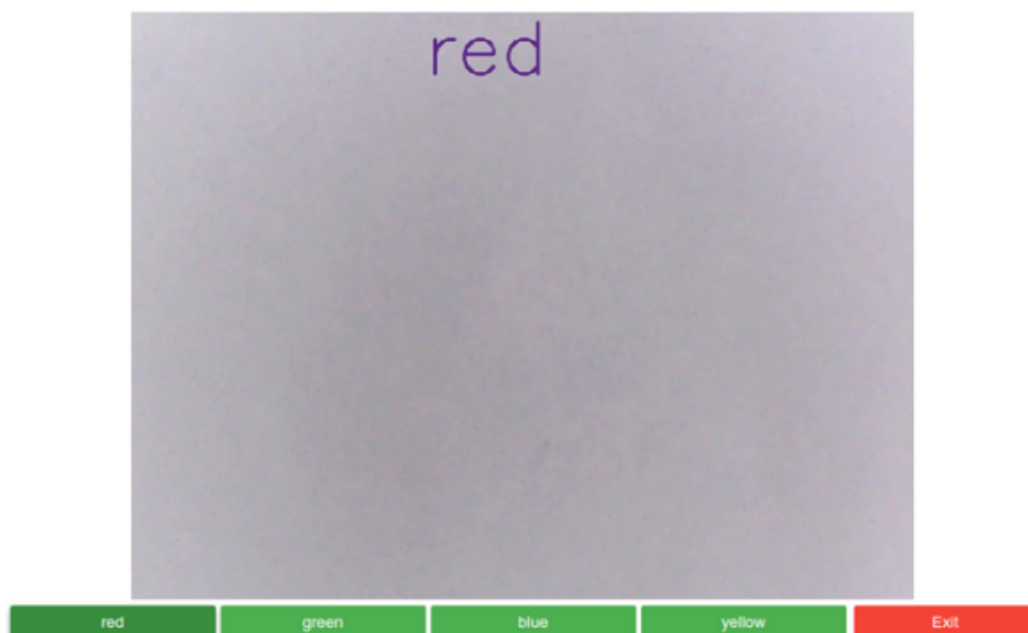
(2) When the selected color appears in the field of view, the robotic arm estimates the position of the block based on the area of the block in the image.

(3) When the area becomes smaller, the robotic arm drives forward until the front completes the grabbing action. When the area becomes larger, it backs away, shaking its head in a fearful state.

(4) When reaching a certain distance, the robotic arm will honk once. At this time, you can place the color block at the gripper. The robotic arm will place the color block at the specified position.

(4) This case relies on HSV for color judgment. When tracking color blocks, keep the background as simple as possible. Pure white is best.

Note: Control the robotic arm back and forth with the color block, then slowly move away to trigger it. This case requires multiple attempts.



Code Design

- Use polygon approximation method to obtain the area of color objects in the field of view

```

for i, cnt in enumerate(contours):
    # Calculate moments of polygon
    mm = cv.moments(cnt)
    cx = mm['m10'] / mm['m00']
    cy = mm['m01'] / mm['m00']
    # Calculate contour area
    area = cv.contourArea(cnt)

```

- Obtain current posture through forward kinematics

```

def get_Posture(self):
    '''
    Publish joint angles, get position
    '''
    self.read_joint()
    # Wait for server to start
    self.client.wait_for_service()
    # Create message packet
    request = kinemaricsRequest()
    ...    ...

```

- Estimate position based on area size

```

# Estimate the distance of the block from the camera
distance = 27.03 * math.pow(area, -0.51)

```

- Inverse solution to obtain the angle each joint needs to rotate

```

def snake_run(self, point_y):
    '''
    Publish position request, get joint rotation angle
    '''
    pass

```